Galactik Wars
A ”MultiPlayer”/”SinglePlayer” third person shooter spaceship game
Segunda Parte

Ricardo Alexandre
Gonçalo Amador
Universidade da Beira Interior

SDTF UBI Technical Report



Junho 2008

Galactik Wars
A ”MultiPlayer”/”SinglePlayer” third person shooter spaceship game
Segunda Parte

Ricardo Alexandre
Gonçalo Amador
Universidade da Beira Interior

Technical Report-2008-10
Junho 2008

Abstract

A tolerância a falhas é uma área crítica num mundo cada vez mais dependente de sistemas informáticos. Neste trabalho abordamos os jogos, a mais abranjente das áreas da informática. Nestes estão presentes campos como Computação Gráfica, Animação e Realidade Virtual, Inteligência Artificial, Teoria da Computação, Segurança Informática, Sistemas Distríbuidos e Tolerância a falhas e Redes. Este trabalho consiste na implementação de um protótipo de jogo em rede (Primeira Parte) e na implementação de técnicas de tolerância a falhas estudadas no decorrer da cadeira (Segunda Parte).

1 Introdução

O objectivo deste trabalho é a implementação de um protótipo de um jogo recorrendo á framework (da Microsoft) XNA, este jogo deverá funcionar em rede (”MultiPlayer” ou ”System Link”) e em modo jogador único (”SinglePlayer”). Note-se que nunca foi o nosso objectivo ter um jogo em ”Single Player” elaborado, apesar de termos adicionado o máximo de funcionalidades possíveis (referidas posteriormente em maior detalhe), não temos nenhuns modos de jogo. Isto verifica-se, pois o pretendido por nós com a parte de ”SinglePlayer” era a aquisição de conhecimentos que necessitávamos para conseguirmos ter a versão ”MultiPlayer” a funcionar. Nesta por sua vez temos um modo de competição simples. Temos portanto um protótipo de um potêncial jogo, uma vez que era inviável criar um jogo elaborado num tão curto espaço de tempo (para se ter uma ideia um jogo de telemóvel leva entre 3 a 4 meses), com a quantidade de pessoas envolvidas (2 pessoas face uma equipa de pelo menos 10 pessoas). Apercebemo-nos na primeira apresentação em que falámos do que pretendia-mos do nosso trabalho que isto não era tão transparente aos nossos colegas como a nós, pelo que referimos fortemente este ponto antes de prosseguir. Ainda a este respeito, a fim de frisar ainda mais esta consideração, nos créditos temos versão ”ALPHA” ao invés de ”BETA” (tendo esse nome sido colocado inicialmente por brincadeira), designação cuja importância constatámos que jamais deve ser levada de animo leve.

Neste relatório indicamos o que obtivemos de entre os objectivos a que nos propusemos originalmente. No modo de jogo em rede acrescentámos com sucesso 3 das 4 técnicas de tolerância a falhas, a que nos propusemos que são pertinentes para um jogo, dentro da categoria em que este protótipo se enquadra (Jogo de acção na terceira pessoa). Este trabalho serviu para nos iniciar na linguagem C# e aplicar os conhecimentos que assimilámos ao longo da licenciatura nas diversas áreas, já referidas (no abstract). No entando de frisar que Inteligência Artificial e Segurança são campos que descurámos, mas que para o resultado deste trabalho se poder designar efectivamente de jogo deverão ser tomados em conta. Nas secções seguintes debruçamo-nos sobre (por esta ordem) as ferramentas utilizadas, as funcionalidades implementadas, a arquitectura e funcionamento do jogo em rede e algumas conclusões que achámos pertinetes de reter.

2 Ferramentas ultilizadas

- .Net Framework 2.0 ou superior - Esta framework da Microsoft é actualmente o pilar base de toda a aplicação desenvolvida em Visual Studio para a Web. Independentemente de se é um jogo, um ”web-service”, etc nesta framework a Microsoft faculta um pacote de métodos e bibliotecas para todas as funcionalidades pertinentes (ou que eles acharam) necessárias para este fim. Suporta todas as linguagens desenvolvidas pela Microsoft (VC++, VB, C#). No nosso caso temos a versão 3.5 que é uma extensão á versão 2.0, que por sua vez extendeu a versão 1.1.
- Visual Studio 8 - IDE para desenvolvimento de código da Microsoft, sobre a qual o XNA corre. Este código pode ser desde aplicações robustas até web-services. Utilizmos a versão completa no nosso caso (uma vez que temos licença), no entanto a versão express (gratuita com algumas limitações) teria sido suficiente (ver quadro comparativo em http://msdn.microsoft.com/en-us/vs2005/aa700921.aspx para uma ideia mais detalhada).
- SpaceWar - Exemplo de aprendizagem de onde retirámos grande parte dos efeitos sonóros e modelos que utilizámos no nosso protótipo de jogo. Distribuido para fins de aprendizagem, não possui modo em rede mas possui ”MultiPlayer”.
- Blender - Ferramenta gratuita para animação e modelação 3D, usámo-la para fazer o modelo do escudo. Esta ferramenta permite exportar em formato .x (DirectX format), que é o formato de modelos 3d que o XNA suporta.

- XNA Framework 2.0 - Framework especificamente criada para o desenvolvimento de jogos para Windows ou XBOX 360, usando a linguagem C#. Desenhada utilizar os recursos do Microsoft Visual C#, esta framework apoia-se sobre a técnologia DirectX e a plataforma .Net. Permite a manipulação/edição de som com uma ferramenta própia (XACT), importação de modelos e texturas, entre outras características pertinentes ferramentas de desenvolvimento de jogos. Estende a plataforma .Net, e tem para os modos de jogo em rede suporte ao serviço de jogos do Live e de sessões "System Link" (ligação entre máquinas dentro de uma rede local). Existe uma alternativa gratuita conhecida por Lindgren, apesar de gratuita ainda não completamente implementada (seria no entanto uma possível tema de estudo ou quem sabe para um projecto).
  O XNA tem uma componente designada de "Content Pipeline", sendo o responsvel por converter (quando é feito o "build") os formatos suportados em formatos reconhecidos durante a execução. Os formatos que podem ser usados são limitados, e no XNA 3.0 (lançamento previsto para o verão de 2008) alguns vão deixar de ser suportados (ainda não estão definidos quais explicitamente). Actualmente são suportados os seguintes formatos:
  - Ficheiros 3D (Modelos): .FBX e .X
  - Ficheiros 2D (Texturas): .BMP, .JPG, .PNG e .TGA
  - Audio: .wav, .aif, .aiff (estes 3 representam os formatos de audio que podemos colocar num projecto do XACT), .XAP (gestor de conteúdo audio nos formatos já indicados XACT)
  - Ficheiros de informação (Materiais e Dados) - .FX e .XML (este segundo através das funcionalidades do Visual C#)

3 Funcionalidades

Tal como já foi referido o nosso objectivo foi criar um protótipo de jogo e apesar de não termos chegádo tão longe como pretendia-mos ultrapassá-mos as nossas expectativas (não temos um demo de jogo, mas as bases para esse objectivo estão asseguradas). De seguida indicamos as funcionalidades, e o respectivo estado de implementação destas. Posteriormente falamos destas mais detalhadamente.

- Funcionalidades Primeira Parte:
  - Sistema de menus. (Implementado)
  - Opções de regulação de som (audio e efeitos). (Implementado)
  - Modo Single Player. (Implementado)
  - Modo ”MultiPlayer”. (Implementado)
  - Fundo móvel e múltiplos cenários de fundo. (Implementado)
  - Sistema de particulas. (Implementado)
  - Câmara genérica. (Implementado)
- Funcionalidades Segunda Parte:
  - Pontuação em jogo ”MultiPlayer”. (Implementado)
  - Migração do Host se este cair. (Implementado)
  - Manutenção do estado de um dos jogadores se este cair, por um tempo mínimo. (Implementado)
  - Permitir a um cliente aderir a uma sessão se esta ainda decorrer. (Implementado)
  - Predicção de enventos. (No Implementada)

3.1 Funcionalidades Primeira Parte

Estas são as bases para termos o nosso (protótipo de) jogo a funcionar, e para futuramente este poder ser estendido (fora do âmbito da cadeira) a um demo funcional e quem sabe a um jogo completo.

3.1.1 Sistema de menus

Existe um sistema de menus já feito para jogos de rede ou só ”SinglePlayer”. Fizemos o download deste, e tentou-se perceber o código o melhor possível, sem documentação, e estendeu-se/alterou-se conforme o pretendido para o caso a aplicar. No nosso caso acrescentámos as secções ”credits” e ”options”. O menu está actualmente implementado e a funcionar, não alteramos o aspecto gráfico deste, somente nos preocupámos em implementar os menus referidos.
A imagem seguinte explica o que é possvel fazer no sistema de menus:

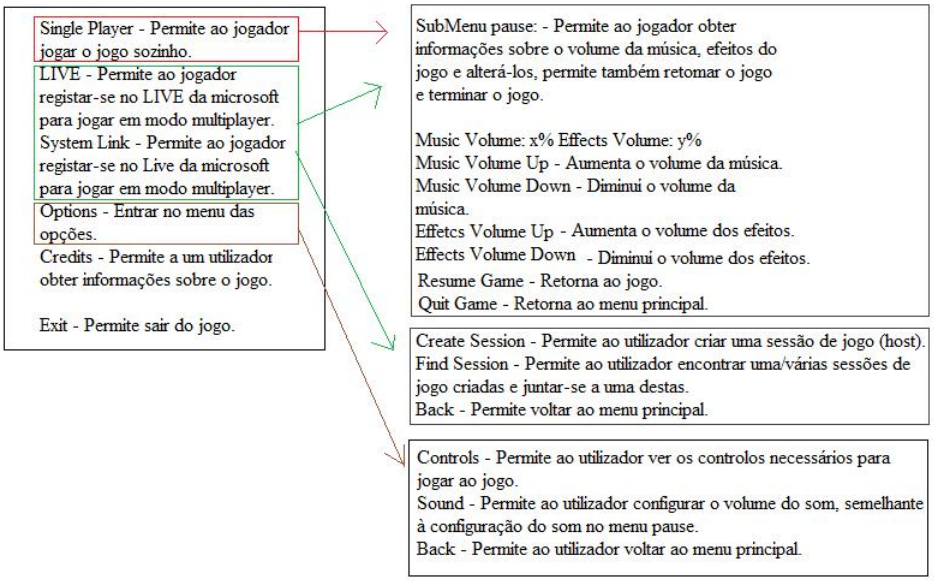


Figure 1: Funcionalidades do Sistema de Menus.

3.1.2 Opções de regulação de som (audio e efeitos)

Dentro do menu por nós adicionado (”options”), podemos regular o som da música de fundo e os efeitos especiais. Isto também pode ser feito nos modos de jogo, quando pausamos o jogo (em rede não pausamos o jogo quando premimos a tecla ”Escape”, tal como nos FPS em rede aparece o menu mas o jogo continua a funcionar, pois não faz sentido um dos clientes poder parar o jogo a todos os outros jogadores por tempo indeterminado), para o efeito acrescentámos uma nova funcionalidade, além das que existiam, quando o jogo se encontra em ”pause”, para regular audio e efeitos sonoros. Adicionámos além dos sons retirados do kit de treino conhecido por ”SpaceWar” (de onde retirámos os nossos modelos e efeitos sonóros) uma compilação de várias músicas para música de fundo.
Algumas notas, no XNA o som é suportado no formato wave outros formatos são utilizados de forma não nativa via ”Content Pipeline”. Este mecanismo funciona como um interpretador de determinados formatos para aúdio (.wav), objectos (.x), texturas (.png, .tga), para um código máquina a ser corrido pela máquina virtual do C#. No nosso trabalho temos uma classe, feita por nós, que tem os métodos necessários a manipulação do som (”SoundHandler.cs”). Temos um bug na manipulação do som na parte de rede que constatámos, para o qual ainda não existe uma solução divulgada, mas que tentámos por várias formas resolver sem sucesso. Este problema devesse a uma limitação na ferramenta de controlo de listas de aúdio (”Cues”) e suas respectivas propriedades (o som toca em loop, nome da faixa, etc) utilizada no XNA (XACT). Estamos no entanto ainda a tentar ver se existe alguma alternativa que não tenhamos experimentado que resolva este problema, motivo pelo qual em rede só a nossa nave faz barulho quando se desloca. De seguida uma pequena amostra de código essencial pertinente para manipular som no XNA:

- Criação dos objetos
  AudioEngine audioEngine;
  WaveBank waveBank;
  SoundBank soundBank;
  protected override void Initialize(){
  audioEngine = new AudioEngine(”MySounds.xgs”);
  waveBank = new WaveBank(audioEngine, ”Wave Bank.xwb”);
  soundBank = new SoundBank(audioEngine, ”Sound Bank.xsb”);
  base.Initialize();
  }
- Gerar um efeito sonoro (som sem loop)
  soundBank.PlayCue(”chord”);
- Tocar música de fundo (som com loop)
  // Iniciar a música
  myLoopingSound = soundBank.GetCue(”notify”);
  myLoopingSound.Play();
  // Parar ou continuar a música
  if (myLoopingSound.IsPaused)
  myLoopingSound.Resume();
  else
  myLoopingSound.Pause();

3.1.3 Modo ”SinglePlayer”

Neste modo seguimos um tutorial online para fazer um jogo tipo ”asteroids”. Após este estar concluído acrescentámos algumas extras (rockets, escudo) e migrámos o código para o nosso sistema de menus (classe própria para o jogo chamada ”GamePlayScreen.cs”). O jogo neste modo tem uma dificuldade crescente, que conciste no aumento da velocidade dos asteroides quando estes são restaurados, após a destruição de todos os asteroides que já existiam (número regulável na classe ”GameVariables.cs”). O jogo acaba se o utilizador após perder o escudo bater num meteoro. O valor inicial do escudo e o dano que cada meteoro causa no jogador são variáveis da classe ”GameVariables.cs”. Quando isto acontece aparace um menu de ”Game Over”, com a respectiva musica de fundo. O jogo tem as bases (referidas de seguida) para ter um sistema de níveis e inimigos elaborado, mas trata-se de um modo relativamente simples actualmente. Temos presente a informação das vidas, escudo restante, asteroides existentes, nível e pontuação actual do jogador.

3.1.4 Fundo móvel e múltiplos cenários de fundo

A ideia por detrás desta funcionalidade prende-se com potencializar níveis dum tipico shooter de naves. A maioria dos tradicionáis jogos 2D de naves em ”SinglePlayer” tem níveis em que o cenário se move. Ao fim de algum tempo aparece um inimigo mais forte (Level Boss ou Level Sub Boss seguido mais a frente do anterior), se este for derrotado passamos de nível. O jogo acaba ao fim de determinada quantidade de níveis, ou se perdermos um determinado número de vidas. Esta funcionalidade (fundo móvel) pode ou não estar activada (controlado pela variavél ”ScrollingBackgroundIsActive” na classe ”GameVariables.cs”). Se esta funcionalidade não estiver activa a classe que implementa esta funcionalidade designada ”ScrollingBackground.cs” não é criada e é feita uma escolha aleatória de entre um conjunto de fundos estáticos (acrescentádos por nós), no arranque do jogo.

3.1.5 Sistema de partículas

Esta funcionalidade pretende simular efeitos naturais como por exemplo explosões: quando a nave for destruida, quando um asteroide explode e quando um rocket explode. Existe uma quantidade considerável de outros efeitos interessantes afim de tornar um jogo mais interessante, sendo esta funcionalidade algo que nunca implementámos mas que já usamos e estudámos (efeitos de partículas é uma das funcionalidades do Blender). É um dos componentes mais comuns dos motores gráficos actuais. Uma vez que esta funcionalidade não é requerida tinhamo-la como opcional, concluimo-la para o modo de singlePlayer. Não temos efeitos 3D elaborados, estes serão uma das melhorias que teremos em conta no processo de transformar este protótipo num demo funcional (ou quem sabe um jogo completo).

3.1.6 Câmara genérica

A câmara que usamos actualmente é estática, ou móvel se o desejarmos (segue a nave no nosso caso). Como é comun em muitos tipos de jogos (se não todos), existe uma classe câmara que permite vizualisarmos o jogo de determinados ângulos pretendidos. De notar que este jogo é em 3D (os modelos são 3D mas os cenários não) mas é visto em 2D (de cima projectado no plano), pelo que o jogo poderia ter uma versão 2D (visto de cima) e 3D (visto a uma pequena distância da parte traseira da nave). A ideia é que o jogo seja em 3D ou 2D quase só pela alteração desta classe (”CCamera.cs”), e com a adição das considerações inerentes a um novo grau de liberdade (movimento dos objectos detecção de colisões, etc). Os controlos estendidos ao 3D são um trabalho que poderá ser feito posteriormente á conclusão da cadeira.

3.1.7 Modo ”MultiPlayer”

Neste modo, no sistema de menus, já tinhamos a parte da criação de sessões tratada; a parte da tolerância a falhas não se encontra aqui tratada, ficando ao nosso critério o que queremos fazer e como o vamos fazer.
O que nós temos é o nosso protótipo de jogo a funcianar em ”MultiPlayer” dentro de uma rede (lan local e não sobre a internet). Temos uma sessão em que quem a cria é o seu Host e os outros utilizadores podem criar a sua sessão ou juntar-se a uma já criada. A rede mais em detalhe é explicada no proximo capítulo. Cada sessão local pode ter no máximo 16 jogadores e uma sessão em rede pode chegar a 31 jogadores no máximo; no nosso caso reduzimos para 6, actualmente o unico motivo para isto é que para mais de 6 temos de ter um mundo com modelos menores que se torna cansativo para a vista.
Neste modo temos uma sessão de até 6 jogadores que lutam entre si o último vivo ganha a sessão. Será mostrada a sua pontuação bem como a dos outros jogadores antes de a sessão terminar.
No jogo temos duas entidades um Host e os clientes. Os passos por que estes passam desde o ínicio ao fim de uma sessão encontram-se de seguida descritos.

Passos no Host:

- Executar sign-in.
- Criar Sessão (abrir um ”Lobby”).
- Esperar que os jogadores se liguem e informem que estão prontos (”ready”).
- Iniciar jogo (sair do ”Lobby” para o jogo).
- Trocar mensagens durante o jogo.
- Terminar jogo (sair do jogo).

Passos nos clientes:

- Executar sign-in.
- Procurar uma Sessão com slots disponíveis.
- Conectar-se á sessão (entrar no ”Lobby”).
- Informar que se está pronto para iniciar (”ready”).
- Confirmar este estado para saber quando jogo começou e terminou.
- Trocar mensagens durante o jogo.

O que tivemos de fazer neste modo para o envio de dados, consistiu em recorrer aos métodos de passagem de mensagens da plataforma .Net (extendidos no XNA ”System.Threading”, ”Microsoft.Xna.Framework.GamerServices” e ”Microsoft.Xna.Framework.Net”), para enviar os dados que cada jogador precisa de saber sobre os restantes. Estes são (neste exemplo reduzido):

- **Envio de Mensagens:**

```
PacketWriter packetWriter = new PacketWriter();
public void SendMessage(string mensagem) {
foreach (LocalNetworkGamer localPlayer in session.LocalGamers) {
packetWriter.Write(mensagem);
localPlayer.SendData(packetWriter,SendDataOptions.None); } }
```

- **Recepção de Mensagens:**

```
PacketReader packetReader = new PacketReader();
public string ReceiveMessage() {
String Mensagem = String.Empty;
NetworkGamer remotePlayer;
foreach (LocalNetworkGamer localPlayer in session.LocalGamers) {
while (localPlayer.IsDataAvailable) {
localPlayer.ReceiveData(packetReader, out remotePlayer);
if (!remotePlayer.IsLocal)
message = packetReader.ReadString(); } } }
```

3.2 Funcionalidades Segunda Parte

Das técnicas de tolerância a falhas pertinetes vistas no decorrer da cadeira, as que se seguem foram as que optámos por implementar.

3.2.1 Pontuação em jogo ”MultiPlayer”

Esta funcionalidade consiste em mostrar a pontuação ordenada dos jogadores (dos que têm mais para os que têm menos pontos). Está implementada bem como a manutenção dos estados durante uma sessão que era requerida, para esta funcionar. Essencialmente se o número de jogadores activos (vivos ou seja com a variavél ”isActive”, da classe ”ship.cs”, a ”true”) for igual ou inferior a um aparecem as pontuações. De notar que as pontuações só são guardadas até ao termino da sessão (quando só estiver um dos jogadores, inicialmente na sessão, vivo ou presente na sessão). Passados 30 segundos, depois de as pontuações aparecerem, aparece o menu de ”pause”.

3.2.2 Migração do Host se este cair

Para aplicar esta funcionalidade temos uma propriedade dos objectos do tipo ”NetworkSession” que assegura que é realizada a migração do Host para um dos clientes presentes na sessão. De notar que o nosso jogo funciona em P2P senão esta funcionalidade seria impraticável (num modelo cliente-servidor o servidor seria substituido por um de backup mas um cliente seria sempre e somente um cliente)! Além de possibilitar a migração de Host tivémos também de garantir a coerência dos dados dos clientes. O que acontece é que quando um jogador cai, todos com ID acima dele descem o seu ID uma unidade (o 1 passa a ser o 0 e assim sucessivamente). Tivemos de assegurar neste caso que somente o Host transmite os dados de todos os jogadores para todos os jogadores na sessão quando um novo jogador entra.

3.2.3 Manutenção do estado de um dos jogadores se este cair, por um tempo mínimo

Nesta funcionalidade se um cliente caiu, o seu estado é mantido deste até que ele retorne ou outro jogador tome o seu lugar. No segundo caso o novo jogadar fica com os dados do jogador que caiu. Até ao fim da sessão estes dados são mantidos. Apesar de isto não ser o que descrevemos originalmente viémos a constactar que é o mais correcto e comun neste tipo de jogos. Neste caso se um jogador cair temos de verificar se o nosso ID foi alterado, se sim temos de comunicar aos outros jogadores para que ajustem os seus dados em conformidade. Cada jogador verifica se o seu e só o seu ID foi alterado e depois envia se for o caso o velho e o novo ID. Cada jogador neste caso é informado que ouve uma alteração e recebe os IDs que deve corrigir de todos para todos, se um jogador entrar ou sair.

3.2.4 Permitir a um cliente aceder a uma sessão se esta estiver a decorrer

Para a funcionalidade anterior ser possível temos de possibilitar a um cliente entrar numa sessão a decorrer. Isto é também uma propriedade, tal como a migração do Host, que temos de activar. O local onde se alteram as duas é no constructor da classe ”NetworkSessionComponent.cs”. De notar que por omissão ambas estão desactivadas. Sempre que um jogador entra numa sessão a decorrer o Host transmite a todos os outros jogadores na sessão os dados de todos os jogadores.

3.2.5 Predicção de enventos

Esta funcionalidade não foi implementada. Essencialmente seria manter durante algum tempo as ultimas acções de um jogador não foi implementada pois passava por usar IA para ser bem efectuada (um sistema de aprendizagem). Devido ao espaço apertado de tempo que tinhamos ao nosso dispor não foi possível.

4 Arquitectura e funcionamento do jogo em rede

Nesta secção falamos em detalhe da estructura do nosso protótipo de jogo em rede e um pouco sobre o funcionamento deste. Desde a arquitectura do referido, ao tipo de protocolos e garantias que este faculta, até em que medida utilizamos a plataforma .Net via XNA. Justificamos ainda o porquê de algumas das escolhas que fizemos, na implementação do modo de rede.

4.1 Arquitectura dum programa em XNA

Antes de prosseguir-mos uma pequena explicação sobre o que é um projecto tipico em XNA. Quando criamos um projecto novo são criados dois ficheiros, sendo eles ”Program.cs” (”Game.cs” no nosso caso) e ”Game1.cs”. O ”Program.cs” corresponde a uma classe ”main” que corre um processo do tipo ”Game1”, sendo o esqueleto do seu método principal algo deste género:

```
static void Main(string[] args)
{
using(Game1 game = new Game1())
{
game.Run(); /*executa o Jogo, até que este termine*/
}
}
```

A classe Game é a classe mais importânte no XNA, um ciclo infinito que (por omissão) é actualizado a cada 1/60 de segundo (a propriedade ”TargetElapsedTime” é a que define este tempo, podendo ser editada no construtor da classe ”Game1.cs”). O constructor desta classe encontra-se explicitamente escrito e tem a seguinte forma mínimal:

```
public Game1()
{
graphics = new GraphicsDeviceManager(this); /* Objecto responsável pela gestão dos gráficos, câmara, etc*/
content = new ContentManager(Services); /*gestor de conteúdos do tipo texturas, modelos 3D, etc. Som não incluido*/
}
```

Exitem cinco métodos prédefinidos na classe ”Game1.cs”, essenciais em qualquer jogos, sendo eles:

- Initialize(): Chamado no acto de carregamento do jogo, neste são inicializados os recursos não gráficos (rato, som, teclado, etc).
- LoadGraphicsContent(bool loadAllContent): Chamado sempre que for necessário carregar todos os conteúdos gráficos. Este método no nosso caso sempre que um ”GamePlayScreen.cs” for criado será chamado.
- UnLoadGraphicsContent(bool unloadAllContent): Chamado sempre que for necessário ”descarregar” todos os conteúdos gráficos. Por vezes utilizado para libertar também recursos inicializados no método initialize (no nosso caso foi verificado que tinhamos de libertar os recursos de audio e foi feito neste método).
- Update(GameTime gameTime): Método onde se encontra a lógica principal do jogo. Chamado a cada ciclo do jogo. No nosso caso verificá-mos que tinha de chamar métodos especificos. Fizemos inclusivé um esforço para ter secções especificas para determinadas funcionalidades, para permitir uma mais rápida alteração e maior legibilidade do código.
- Draw(GameTime gameTime): Método responsável por desenhar os elementos gráficos do nosso jogo (objectos, texturas, etc). Chamado a cada ciclo do jogo. No nosso caso temos extensões métodos que este chama que se destinam ao desenho de jogadores, modelos e tiros (balas e misseis).

A grande diferença no nosso caso face a esta estructura base, passa pelo facto de termos um sistema de menus. O que temos de distinto é então que ao invés de um classe jogo temos um gestor de menus. Este gere onde estamos os menus que tem de ser criados, destruidos, recriados. O nosso jogo passa a ser um tipo particular de menu. Temos de recorrer ao mecanismo de errança, para desta forma obter os valores dos atributos necessários no constructor do jogo.

4.2 P2P vs Cliente Servidor

O nosso jogo poderia seguir dois caminhos ou seguiamos uma arquitectura Cliente Servidor ou P2P.

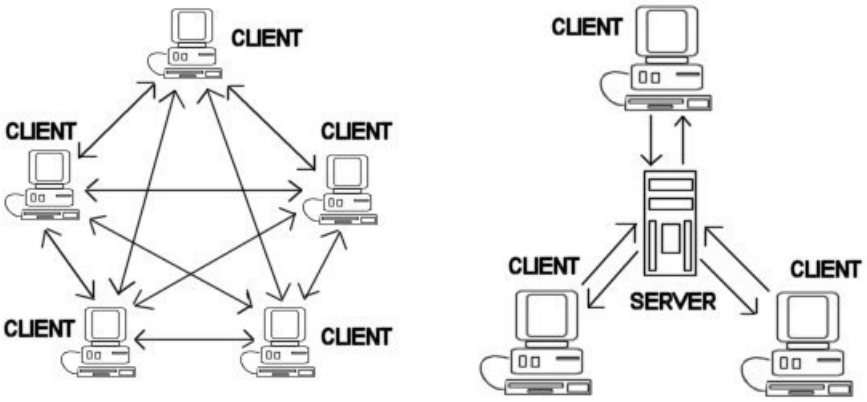


Figure 2: Arquitecturas P2P e Cliente-servidor (respectivamente)

A nossa opcção remeteu para o P2P (de referir ambas eram aplicáveis, tinhamos inclusivé exemplos com código de implementação das duas). O porquê liga-se com o facto de as redes P2P serem um dos sentidos evolutívos das redes informáticas. Os jogos acompanham essa evolução sendo emergente cada vez mais jogos em P2P, face á arquitectura cliente servidor, para o nosso tipo de jogo (Shooters em tempo real FPS por exemplo). Os jogos que funcionam em P2P aproveitam melhor os recursos (apesar de ocuparem maior largura de banda) da rede e locais, e são mais tolerantes a falhas. Perdemos no entanto a possíbilidade de implementar uma considerável quantidade de técnicas de tolerância a falhas, que seriam necessárias implementar na arquitectura cliente servidor, por outro lado ganhamos em tolerância a falhas nomeadamente quando o servidor cái. Neste caso temos um cliente a tomar o seu lugar, conseguimos evitar desta forma a redundância de servidores de backup. As redes P2P são também muito mais simples de implementar (a estructura da rede em si).

4.3 Comunicação durante o jogo em rede

A ter em conta que o nosso jogo funciona em P2P em termos de comunicação. Cada jogador gere o seu input, envia os seus dados a todos os outros jogadores e estes actualizam as suas cópias locais deste. Ou seja cada jogador recebe os dados de todos os outros jogadores, actualiza as sua cópias locais dos mesmos, envia os seus dados a todos os outros e desenha tudo (texturas, modeles, etc).

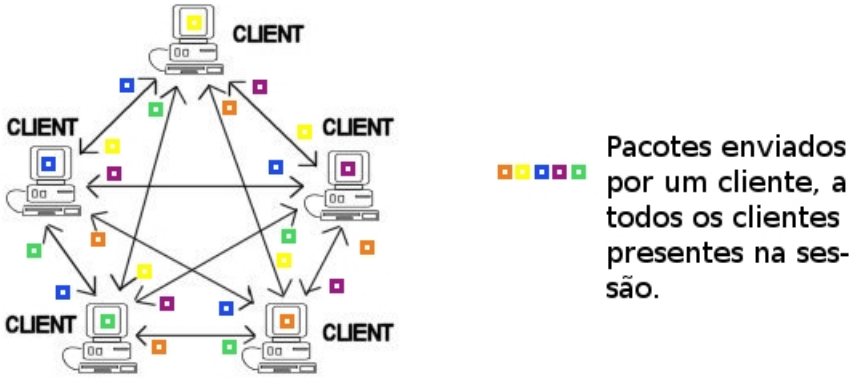


Figure 3: Comunicação durante o jogo

Os dados enviados por cada cliente passaram a ser booleanos permitindo uma comunicação mais rápida, e um acréscimo na segurança. Este último verifica-se pois algúem mal intencionado apenas sabe que nos estamos ou não a deslocar, ou a disparar mas não tem forma de saber a nossa última posição (a menos que se aproveite da tolerância a falhas quando um jogador entra). Os dados que são enviados aos restantes, se não houver nenhuma falha, são:

- bool isActive: indica se a nossa nave está activa (estamos vivos?).
- float Radius: indica o raio da nossa nave (será diferente consoante temos ou não escudo).
- bool isFiringBullets: indica se estamos a disparar balas.
- bool isFiringRockets: indica se estamos a disparar misseis.
- bool isRotatingLeft: indica se estamos a rodar para a esquerda.
- bool isRotatingRight: indica se estamos a rodar para a direita.
- bool isMovingDown: indica se nos estamos a deslocar para trás.
- bool isMovingUp: indica se nos estamos a deslocar para a frente.

Tal como tinhamos referido na primeira parte podiamos emagrecer a comunicação enviado booleanos ao invés da velocidade e posição. Tornámos desta forma o nosso jogo mais seguro e eficiênte.

Não temos no nosso caso um servidor central a tratar dos dados de todos e a enviá-los a cada um dos clientes. O Host é um ”servidor de sessão”, mas não é fixo, se ele perder a ligação outro cliente toma o seu lugar. O que cada jogador tem é um array com uma cópia dos dados dos outros jogadores (posição, escudo, pontos, etc). Temos então aqui a redundância que vamos necessitar de guardar para garantir a manutenção do estado de um jogador durante um determinado intervalo, se este cair por falha na rede.

4.4 Protocolos e QoS

Os nossos pacotes são UDP com garantia de recepção por ordem de envio (”SendDataOptions.ReliableInOrder” no ”GamePlayScreen.cs”). A razão de utilizarmos UDP prende-se com ser o que é recomendado em jogos de acção, como é o nosso ou dos FPS”. Isto porque UDP é rápido e pequenas perdas são preferiveis a tempos de espera indefinidos, resolvendo-se a perda de pacotes com a implementação de sistemas predictivos como técnica de tolerância a falhas. A qualidade de serviço é assegurada indicando o tempo de médio de ”Round trip” nas sessões existentes (”qualityOfService.AverageRoundtripTime” na classe ”AvailableSessionMenuEntry.cs”), podendo desta forma o cliente optar por se juntar a sessões com melhor tempo de resposta (esta opcção é bastante comun em jogos em rede). Como não temos o sistema preditivo e precisamos de garantir, no caso do envio dos dados se alguém reentrar, por ordem.

4.5 Uso da Plataforma .Net e Xna

A plataforma .Net faculta os mecânismos de Threading (as APIs já foram referidas anteriormente), e a plataforma XNA estende as funcionalidades da plataforma .Net ás necessidades especificas de jogos. Em caso de queda de um Host é referido que é escolhido dos menbros dessa sessão um para tomar o lugar de Host, mas nunca é especificado como é feita a escolha (é algo interno não indicado, possívelmente o com melhor ligação uma vez que o Xna permite analizar a qualidade da rede). A funcionalidade de rede funciona através do serviço Live da Microsoft que entre outras funcionalidades permite sessões de jogos online ou em rede local. O XNA utiliza ainda um serviço de proxy (XnaLiveProxy), possívelmente para dar garantias de segurança (ocultar a identidade dos intervinientes na sessão). A biblioteca primordial que aqui usamos designase por ”Microsoft.Xna.Framework.Net”, esta é responsável pelo suporte para Xbox LIVE, multiplayer, e networking. A sua descrição pode ser consultada em http://msdn.microsoft.com/en-us/library/microsoft.xna.framework.net.aspx.

5 Conclusões

O XNA é uma framework muito completa para desenvolver jogos, no entanto, ainda com alguns pontos fracos muito críticos:

- Não suporta OpenGL;
- Não tem um editor de modelos nativo;
- Não permite multi-formatos, tendo de ser convertidos na maioria das vezes por nós para o som, texturas e modelos;
- O XACT em rede apresenta algumas lacunas não resolvidas; (no entanto em julho saí o XNA 3.0)
- Não (aparentemente) permite gerar executáveis para outras plataformas (Sistemas Operativos);
- Não é possível correr em computadores sem as bibliotecas do XNA.

Existem numerosos exemplos de código mas a sua documentação é inexistente e a maioria das vezes a integração ou uso destes passa pela sua rescrita o que pretendemos.
O .Net torna a criação de aplicações para rede muito mais simples de forma rápida.
Motores de particulas são componentes elaboradas, razão pela qual são tema constante de estudo.
Sistemas de tolerância a falhas embora simples de perceber são de delicada (dificeis e muitas considerações e comprimissos são feitos) implementação.